

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL
INGENIERÍA DE SOFTWARE
APLICACIONES WEB — 5.º NIVEL

PROYECTO FIN DE CURSO — ENTREGA 1B
SGED — Sistema de Gestión para Escuela Deportiva

Módulo de Autenticación JWT + Acceso a Datos con Spring Data JPA

Campo	Detalle
Integrantes	Arcalle Grefa Darwin Orlando Pallo Pinto Daniel Alejandro Vélez López Ricardo Enrique
Docente	GUERRERO ULLOA GLEISTON CICERON
Asignatura	Aplicaciones Web — 5.º Nivel
Entrega	1B — Semana 7 — 2025-2026
Repositorio Git	https://github.com/DarwinSM21/SGED_APPWEB.git
Rama de entrega	entrega-1b
Tag de entrega	v0.1.0-entrega-1b
Fecha	Semana 7 — 6 de junio de 2026

1. Resumen Ejecutivo

El presente documento resume el desarrollo técnico de la primera fase del proyecto SGED, un sistema web de gestión integral diseñado para centralizar la administración de una escuela deportiva, abarcando estudiantes, representantes, entrenadores, asistencia, entrenamientos, equipos, partidos e inventario. En esta etapa inicial se solucionó la vulnerabilidad existente en el acceso a los datos y la falta de una arquitectura base para el control de usuarios. Para su construcción se empleó una arquitectura modular compuesta por un backend en Java 24 con Spring Boot 4.x y Spring Security 7.x, conectado a una base de datos PostgreSQL alojada en Supabase y organizada en los esquemas seguridad, academico, deportivo, inventario y auditoria, mientras que el frontend se estructuró con Angular 21.2.5 mediante componentes Standalone (sin AppModule) y Bootstrap.

Los resultados concretos de esta entrega abarcaron el diseño completo del modelo de datos en PostgreSQL con claves primarias y foráneas, índices, soft delete mediante el campo activo y timestamps de auditoría (created_at, updated_at); la implementación del módulo de autenticación propio (sin usar Supabase Auth) basado en JWT firmado y contraseñas cifradas con BCrypt; y la creación de la estructura modular del frontend en Angular (auth, core, layout, features). Se decidió deliberadamente no implementar mecanismos adicionales como Redis, refresh tokens o Docker, ya que la guía del curso no exige replicar literalmente dichos componentes, sino documentar lo realmente construido. El login fue probado de extremo a extremo con el usuario administrador del sistema, y las rutas protegidas mediante Spring Security respondieron correctamente tanto en ausencia como en presencia de un token válido.

2. Introducción

2.1 Contexto y Motivación

Ecuador cuenta con una amplia red de escuelas de fútbol formativas registradas ante la Federación Ecuatoriana de Fútbol, las cuales atienden a una numerosa población de niños y jóvenes en edad escolar. Sin embargo, la gran mayoría de estas instituciones gestiona su operación mediante registros físicos o planillas de cálculo manuales. Esta situación genera una carga administrativa significativa que resta horas de trabajo semanales a los entrenadores, eleva las tasas de morosidad en el cobro de membresías debido a la falta de un seguimiento automatizado, y provoca una ausencia total de trazabilidad en el rendimiento deportivo de los jugadores. El Sistema de Gestión para Escuela Deportiva (SGED) nace como respuesta a esta brecha tecnológica: una plataforma web de gestión integral que digitaliza el registro de estudiantes y representantes, el control de asistencia, la planificación de entrenamientos, la gestión de equipos y partidos, y el control de inventario deportivo.

2.2 Objetivos de la Entrega 1B

- Diseñar e implementar el modelo de base de datos en PostgreSQL (Supabase), organizado en los esquemas seguridad, academico, deportivo, inventario y auditoria, con claves primarias y foráneas, índices, soft delete y timestamps de auditoría.
- Implementar el módulo de autenticación stateless con JWT propio, sin depender de Supabase Auth, garantizando un flujo correcto de inicio de sesión y validación de identidad mediante Spring Security.

- Configurar Spring Security 7.x para proteger las rutas del backend mediante roles almacenados en base de datos (tabla seguridad.usuario_rol), dejando abiertos únicamente los endpoints públicos de autenticación y pruebas.
- Crear la estructura modular del frontend en Angular 21.2.5 utilizando componentes Standalone (sin AppModule), con las carpetas auth, core, layout y features, e instalar Bootstrap como base visual.
- Verificar el correcto funcionamiento del login y de las rutas protegidas mediante pruebas manuales sobre los endpoints /api/auth/login, /api/test/public y /api/test/private.

2.3 Alcance

La presente entrega cubre el modelado completo de la base de datos y el módulo de Autenticación, junto con la configuración de la estructura base del sistema SGED tanto en el backend como en el frontend. Quedan para las etapas posteriores del proyecto los módulos funcionales de Estudiantes, Asistencia, Entrenamientos, Equipos, Partidos, Inventario y Auditoría desde la perspectiva de interfaz de usuario. En el frontend, a la fecha de esta entrega se ha completado la estructura de carpetas, el guard y el interceptor, y se encuentra en construcción la integración del formulario de login con el backend (consumo de POST /api/auth/login, almacenamiento del JWT y redirección al dashboard); las demás pantallas funcionales se integrarán de forma progresiva en las siguientes fases del cronograma.

2.4 Estructura del Documento

El informe se organiza en las siguientes secciones: Marco Teórico (sección 3) con los fundamentos académicos de las decisiones técnicas; Diseño del Sistema (sección 4) con el modelo de base de datos, la arquitectura general y el flujo de autenticación JWT; Implementación (sección 5) con la estructura del repositorio, la configuración de Spring Security y los endpoints desarrollados; Pruebas (sección 6) con la verificación manual de los flujos de autenticación; Seguridad (sección 7); Decisiones de Arquitectura (sección 8); Estado Actual y Trabajo Pendiente (sección 9); Conclusiones (sección 10); y Referencias (sección 11).

3. Marco Teórico

3.1 Autenticación Stateless vs. Stateful

La autenticación constituye uno de los componentes fundamentales de cualquier aplicación web, ya que permite verificar la identidad de los usuarios antes de conceder acceso a recursos protegidos. Tradicionalmente, las aplicaciones empresariales implementaban un modelo stateful, en el cual el servidor mantenía información sobre cada sesión iniciada mediante un identificador enviado al navegador como cookie. Este mecanismo presenta limitaciones cuando las aplicaciones deben escalar horizontalmente, ya que todas las peticiones de un usuario deben dirigirse al mismo servidor o compartir un repositorio común de sesiones (Richardson, 2019).

Como alternativa surgió el modelo stateless, cuyo principio consiste en eliminar el almacenamiento de sesiones del lado del servidor: cada petición HTTP contiene toda la información necesaria para identificar y autorizar al usuario. El mecanismo más difundido para implementar este modelo es JSON Web Token (JWT), especificado en el estándar RFC 7519 (Jones, Bradley & Sakimura, 2015). Un JWT almacena un conjunto de reclamaciones (claims), como el nombre de usuario, el rol y la fecha de expiración, protegidas mediante una firma criptográfica que garantiza su integridad. Gracias a ello, el servidor únicamente necesita verificar

la firma del token para autenticar la solicitud, sin mantener información adicional sobre la sesión, lo que se ajusta directamente a la decisión de SGED de implementar JWT propio en lugar de delegar la autenticación a Supabase Auth.

El modelo stateless se integra de forma natural con arquitecturas REST y aplicaciones SPA (Single Page Application), como la desarrollada en Angular para SGED, donde el frontend y el backend funcionan como componentes independientes que intercambian información mediante servicios HTTP (Hardt, 2012).

3.2 Spring Security y el Modelo de Filtros

En el ecosistema de desarrollo Java, Spring Security constituye el framework de referencia para implementar autenticación, autorización y protección frente a múltiples amenazas de seguridad. La configuración de seguridad se basa en el componente `SecurityFilterChain`, el cual permite definir de forma explícita la secuencia de filtros que procesan cada solicitud HTTP antes de que esta alcance los controladores de la aplicación.

El funcionamiento interno del framework sigue el patrón Chain of Responsibility: cada filtro ejecuta una tarea específica dentro del proceso de autenticación. En SGED se incorporó un filtro personalizado, `JwtAuthenticationFilter`, registrado antes de `UsernamePasswordAuthenticationFilter` mediante `addFilterBefore()`, con el propósito de extraer el token desde la cabecera `Authorization`, validar su firma y reconstruir el contexto de seguridad correspondiente al usuario autenticado (Walls, 2016).

Para recuperar la información de los usuarios desde la base de datos, se implementó `CustomUserDetailsService`, que mediante `UsuarioRepository` obtiene los datos del usuario almacenados en el esquema seguridad y construye un objeto `UserDetails` con el nombre de usuario, la contraseña cifrada con `BCrypt` y el rol asignado a través de la tabla `seguridad.usuario_rol`. Esta separación de responsabilidades entre `SecurityConfig`, `JwtService`, `JwtAuthenticationFilter` y `CustomUserDetailsService` facilita el mantenimiento del código y permite adaptar la infraestructura de autenticación sin alterar la lógica funcional de la aplicación.

3.3 ORM y Mapeo Objeto-Relacional con Spring Data JPA

El desarrollo de aplicaciones empresariales orientadas a objetos presenta el desafío conocido como impedancia objeto-relacional, que surge porque los lenguajes de programación representan la información mediante objetos mientras que los sistemas gestores de bases de datos utilizan tablas, filas y relaciones. Para resolver esta dificultad surgieron los Object-Relational Mapping (ORM); en el ecosistema Java, Jakarta Persistence (JPA) define la abstracción estándar y Hibernate constituye su implementación de referencia utilizada por Spring Data JPA.

En SGED, cada esquema de la base de datos (seguridad, academico, deportivo, inventario y auditoria) se mapea mediante entidades JPA anotadas con `@Entity` y repositorios que extienden `JpaRepository`, evitando la escritura manual de sentencias SQL para las operaciones más comunes. Yan, Cheung y Lu (2016) señalan que la mayoría de los problemas de rendimiento asociados a un ORM no provienen de la herramienta en sí, sino de decisiones de diseño relacionadas con consultas redundantes o un uso ineficiente del modelo de persistencia, por lo que el diseño cuidadoso de relaciones y soft delete realizado en esta entrega busca anticipar dichos problemas.

3.4 Seguridad de Contraseñas y Tokens

La Open Worldwide Application Security Project (OWASP) publica periódicamente el OWASP Top 10, documento que identifica las vulnerabilidades más frecuentes en sistemas web. La categoría A07: Identification and Authentication Failures engloba debilidades relacionadas con la autenticación de usuarios y la protección de credenciales (OWASP Foundation, 2025).

En SGED, las contraseñas se almacenan exclusivamente como hash mediante BCrypt, algoritmo adaptativo que incorpora un factor de costo configurable y resistencia a ataques de fuerza bruta. El JWT emitido incluye claims mínimos de identificación (sub, rol, iat, exp) firmados criptográficamente, conforme al estándar RFC 7519, de modo que el servidor pueda verificar la integridad y la vigencia del token sin mantener sesiones activas. Para esta entrega se mantuvo deliberadamente fuera del alcance la incorporación de mecanismos adicionales como listas negras de tokens o refresh tokens, priorizando primero un flujo de login y verificación de roles completamente funcional.

4. Diseño del Sistema

4.1 Arquitectura General

El sistema SGED se construyó siguiendo una arquitectura cliente-servidor desacoplada, compuesta por tres capas principales:

Capa	Tecnología	Puerto	Responsabilidad
Frontend	Angular 21.2.5 (Standalone) + Bootstrap	4200 (dev)	SPA; formulario de login; guards e interceptores HTTP
Backend	Java 24 + Spring Boot 4.0.6 + Spring Security 7	8080	API REST; autenticación JWT; lógica de negocio; Spring Data JPA
Base de datos	PostgreSQL (Supabase)	5432	Persistencia relacional en esquemas seguridad, academico, deportivo, inventario, auditoria

4.2 Modelo de Base de Datos por Esquema

La base de datos se organizó en cinco esquemas independientes para separar responsabilidades:

Esquema	Tablas
seguridad	estados_general, personas, roles, usuarios, usuario_rol
academico	categorias, estudiantes, representantes, estudiante_representante
deportivo	estado_asistencia, sesion_entrenamiento, asistencia, ejercicios, entrenamiento_ejercicios, equipos, partidos, estadistica_partidos
inventario	estado Equipamiento, equipamiento, estado_prestamo, prestamo_equipamiento
auditoria	auditoria_global

Cada tabla cuenta con claves primarias y foráneas correctamente definidas, índices sobre las columnas de búsqueda frecuente, un mecanismo de soft delete mediante el campo activo (en lugar de eliminación física de registros) y columnas de auditoría `created_at` y `updated_at`. Los catálogos base (estados, roles, categorías) se precargaron mediante sentencias `INSERT` al momento de crear el esquema.

4.3 Usuario Administrador y Estructura de Seguridad

Para permitir las primeras pruebas de extremo a extremo se creó un usuario administrador en el esquema seguridad, compuesto por una persona (nombre: Administrador, apellido: Sistema), un usuario (username: admin) y la asignación del rol ADMINISTRADOR mediante la tabla `usuario_rol`. Durante la verificación inicial se detectó que el hash BCrypt almacenado no coincidía con la contraseña esperada; tras corregir la generación del hash, las credenciales admin / admin123 quedaron funcionando correctamente contra el endpoint de login.

4.4 Flujo de Autenticación JWT

El flujo implementado para el inicio de sesión sigue los siguientes pasos:

Paso	Origen	Destino	Acción
1	Cliente (Angular)	AuthController	POST /api/auth/login con username y password
2	AuthController	AuthService	Delega la validación de credenciales
3	AuthService	CustomUserDetailsService / UsuarioRepository	Busca el usuario por username en seguridad.usuarios
4	AuthService	PasswordEncoder (BCrypt)	Compara la contraseña ingresada con el hash almacenado
5	AuthService	JwtService	Genera el token firmado con claims sub, rol, iat, exp
6	AuthController	Cliente (Angular)	Responde con token, username, nombre y rol
7	JwtAuthenticationFilter (peticiones posteriores)	SecurityContextHolder	Valida el token y restablece el contexto de seguridad antes de llegar al controlador

4.5 Estructura Modular del Frontend (Angular Standalone)

El frontend no utiliza AppModule; la aplicación se configura íntegramente mediante `app.config.ts` y `app.routes.ts`, siguiendo el modelo de Standalone Components introducido en versiones recientes de Angular. La estructura de carpetas acordada por el equipo es la siguiente:

```
src/app/
├── auth/
│   ├── pages/login/
│   ├── services/auth.service.ts
│   └── models/login-request.ts, login-response.ts
└── core/
```

```

├── guards/auth-guard
├── interceptors/jwt-interceptor
├── services/
├── layout/
│   ├── navbar/
│   ├── sidebar/
│   └── dashboard-layout/
├── features/
│   └── dashboard/pages/dashboard/

```

Esta organización separa: auth (todo lo relacionado con autenticación), core (guards, interceptores y servicios globales reutilizables), layout (elementos visuales compartidos como navbar, sidebar y el layout del dashboard) y features (módulos funcionales independientes, como dashboard, estudiantes, entrenadores, asistencia, inventario y reportes, que se irán incorporando en próximas entregas).

5. Implementación

5.1 Tecnologías Utilizadas

Componente	Detalle
Backend	Java 24, Spring Boot 4.x, Spring Security 7.x, Spring Data JPA, JWT (io.jsonwebtoken), Lombok, Maven
Frontend	Angular CLI 21.2.5, Standalone Components, Node.js 24.12.0, npm 11.6.2, Bootstrap
Base de datos	PostgreSQL alojado en Supabase
Autenticación	JWT propio + Spring Security; contraseñas cifradas con BCrypt

5.2 Estructura del Paquete de Autenticación (Backend)

El módulo de autenticación se organizó dentro del paquete `org.uteq.backend.auth`, siguiendo una separación por capas:

```

auth/
├── controller/    → AuthController
├── service/       → AuthService, JwtService
├── repository/    → UsuarioRepository
├── dto/           → LoginRequest, LoginResponse
├── entity/
├── security/      → JwtAuthenticationFilter, SecurityConfig, CustomUserDetailsService

```

5.3 DTOs de Autenticación

```

// LoginRequest
{
  "username": String,
  "password": String
}

// LoginResponse
{
  "token": String,
  "username": String,

```

```
"nombre": String,  
"rol": String  
}
```

5.4 Endpoint de Login

El endpoint principal del módulo de autenticación es el siguiente:

```
POST /api/auth/login  
  
// Request  
{  
  "username": "admin",  
  "password": "admin123"  
}  
  
// Response  
{  
  "token": "...",  
  "username": "admin",  
  "nombre": "Administrador Sistema",  
  "rol": "ADMINISTRADOR"  
}
```

5.5 Estructura del Token JWT

Claim	Valor de ejemplo	Descripción
sub	"admin"	Nombre de usuario — sujeto del token
rol	"ADMINISTRADOR"	Rol asignado en seguridad.usuario_rol
iat	timestamp Unix	Issued at — momento de emisión
exp	timestamp Unix	Expiration — momento de expiración

El token se genera correctamente y se valida correctamente en cada petición protegida mediante el filtro `JwtAuthenticationFilter`.

5.6 Configuración de Spring Security

La configuración actual en `SecurityConfig` define explícitamente las rutas públicas y protegidas:

```
// Rutas permitidas sin autenticación  
/api/auth/**  
/api/test/public  
  
// Cualquier otra ruta requiere autenticación  
.anyRequest().authenticated()  
  
// Filtro JWT registrado antes del filtro estándar de Spring Security  
.addFilterBefore(  
    jwtFilter,  
    UsernamePasswordAuthenticationFilter.class  
)
```

5.7 Tabla de Endpoints Implementados

Método	URL	Auth	Descripción
POST	/api/auth/login	No	Autentica al usuario y emite el JWT
GET	/api/test/public	No	Endpoint público de prueba; responde "Publico"
GET	/api/test/private	JWT Bearer	Endpoint protegido de prueba; responde "Privado" únicamente con token válido

5.8 Configuración del Frontend Angular

Se instaló Bootstrap como base visual del proyecto:

```
npm install bootstrap
```

Se creó el archivo de entorno con la URL base del backend:

```
// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/api'
};
```

Y se configuraron las rutas iniciales de la aplicación:

```
/login
/dashboard
```

6. Pruebas

Para esta entrega se realizaron pruebas manuales sobre los endpoints de autenticación, verificando tanto el acceso público como el acceso protegido por JWT:

Caso de prueba	Resultado esperado	Resultado obtenido
GET /api/test/public sin token	200 OK con respuesta "Publico"	PASS — Publico
GET /api/test/private sin token	403 Forbidden	PASS — 403 Forbidden
GET /api/test/private con token válido	200 OK con respuesta "Privado"	PASS — Privado
POST /api/auth/login con admin / admin123	200 OK con token, username, nombre y rol	PASS

Estas pruebas confirman que el flujo de autenticación, la generación y validación del JWT, y la protección de rutas mediante Spring Security funcionan correctamente para el usuario administrador creado en el esquema seguridad. Las pruebas automatizadas (JUnit) y las métricas de rendimiento se incorporarán en una etapa posterior, una vez se complete la integración del frontend con el backend.

7. Seguridad Implementada

Control	Implementación	Justificación
Hash de contraseñas	BCryptPasswordEncoder en SecurityConfig	Evita almacenar contraseñas en texto plano (OWASP A02)
JWT firmado	io.jsonwebtoken; claims sub, rol, iat, exp	Autenticación stateless verificable sin almacenar sesiones (RFC 7519)
Roles en base de datos	Tabla seguridad.usuario_rol	Permite gestionar permisos sin modificar código (RBAC)
Rutas protegidas	SecurityConfig + JwtAuthenticationFilter	Solo /api/auth/** y /api/test/public quedan abiertas; el resto exige autenticación (OWASP A01 / A07)
Soft delete y auditoría	Campo activo + created_at / updated_at en todas las tablas	Trazabilidad de cambios sin pérdida de información histórica

Cabe señalar que, por decisión explícita del equipo, no se implementaron en esta entrega mecanismos adicionales como autenticación mediante Supabase Auth, listas negras de tokens (blacklist), refresh tokens o contenedores Docker, ya que la guía del proyecto no exige replicar literalmente todos esos ejemplos. La sección de implementación se redactó basándose en lo realmente desarrollado: Spring Boot, PostgreSQL, JWT, Angular, arquitectura modular y seguridad.

8. Decisiones de Arquitectura

- No usar Supabase Auth: se decidió mantener PostgreSQL de Supabase únicamente como motor de base de datos, implementando JWT y Spring Security propios para tener control total sobre el flujo de autenticación y los roles.
- Roles almacenados en base de datos: la tabla seguridad.usuario_rol permite asociar uno o varios roles a cada usuario sin necesidad de modificar el código fuente.
- Arquitectura modular en el frontend: separación clara entre auth, core, layout y features para facilitar el crecimiento del proyecto en las siguientes entregas.
- Angular Standalone: no existe AppModule; toda la configuración se centraliza en app.config.ts y app.routes.ts, siguiendo el enfoque recomendado en las versiones recientes de Angular.
- Alcance ajustado a la guía: tras analizar la guía de proyecto (SGED.pdf), se concluyó que no es necesario implementar literalmente todos los ejemplos propuestos (Redis, Refresh Tokens, Docker, Flyway, etc.), sino documentar fielmente la arquitectura y las tecnologías efectivamente utilizadas.

9. Estado Actual y Trabajo Pendiente

9.1 Backend

- PostgreSQL configurado en Supabase con los cinco esquemas del sistema.
- Entidades y repositorios JPA implementados.

- Autenticación JWT funcionando de extremo a extremo.
- Contraseñas cifradas con BCrypt.
- Endpoint de login funcionando.
- Spring Security configurado con rutas públicas y protegidas.
- Roles validados a través de seguridad.usuario_rol.

9.2 Frontend

- Proyecto Angular creado y Bootstrap instalado.
- Estructura modular (auth, core, layout, features) creada.
- Guard y interceptor creados (pendientes de conectar al backend).
- Componentes de Dashboard y Login generados.

9.3 Pendiente para la siguiente entrega

- Configurar provideHttpClient() en app.config.ts.
- Implementar auth/services/auth.service.ts para consumir POST /api/auth/login.
- Conectar el LoginComponent con el backend, almacenar el JWT en localStorage y redirigir al Dashboard.
- Implementar el JwtInterceptor para adjuntar el token a las peticiones salientes.
- Implementar el AuthGuard funcional para proteger rutas en el frontend.
- Crear el endpoint GET /api/auth/me.
- Iniciar el desarrollo del módulo de Estudiantes.

9.4 Historial de git

The screenshot displays the GitHub 'Commits' page for the 'main' branch. The interface is dark-themed. At the top, there are filters for 'All users' and 'All time'. The commits are grouped by date, with the most recent group being 'Commits on Jun 20, 2026'. The commits listed are:

- Merge pull request #2 from DarwinSM21/feature/backendAuth** (Verified, 9fbd964) by Alejandro-hub19, 3 hours ago.
- Creacion de Login** (df3bfe6) by Alejandro-hub19, 3 hours ago.
- fix: configurar appconfig v1** (73c0d16) by Alejandro-hub19, 3 days ago.
- Crear estructura backend/frontend e implementar Auth** (c2e3362) by DarwinSM21, 3 days ago.
- docs: Agregar documento** (4e99026) (Verified) by DarwinSM21, 2 weeks ago.
- Add Entrega1A planning and design DOCX** (033b119) by Richfido, 2 weeks ago.
- Docs: Agregar script DB** (c03f007) (Verified) by Alejandro-hub19, 2 weeks ago.
- docs: Agregar archivo README.md del proyecto** (51f1efaa) by DarwinSM21, 2 weeks ago.

Below these, there is a section for 'Commits on May 7, 2026' which is currently empty.

10. Conclusiones

Objetivo 1 — Modelo de base de datos

El modelo de base de datos fue diseñado e implementado exitosamente en PostgreSQL sobre Supabase, organizado en cinco esquemas independientes que reflejan los dominios funcionales del sistema (seguridad, académico, deportivo, inventario y auditoría). El uso de claves primarias y foráneas bien definidas, índices, soft delete mediante el campo activo y columnas de auditoría sienta una base sólida para los módulos funcionales que se desarrollarán en las próximas entregas.

Objetivo 2 — Autenticación stateless con JWT propio

El módulo de autenticación fue implementado exitosamente sin depender de Supabase Auth, cubriendo el flujo completo de login, generación y validación del JWT, y protección de rutas mediante Spring Security. El usuario administrador creado en el esquema seguridad permitió validar de extremo a extremo el correcto funcionamiento del sistema, incluyendo la corrección oportuna de un hash BCrypt que inicialmente no coincidía.

Objetivo 3 — Estructura modular del frontend

La estructura del proyecto Angular se definió siguiendo el enfoque Standalone Components, sin AppModule, separando claramente las responsabilidades en auth, core, layout y features. Esta organización facilitará la incorporación progresiva de los módulos funcionales restantes.

Objetivo 4 — Alcance ajustado a lo realmente construido

Tras revisar la guía del proyecto, el equipo decidió documentar y construir únicamente las tecnologías efectivamente necesarias para el alcance de esta entrega (Spring Boot, PostgreSQL, JWT, Angular, arquitectura modular y seguridad), evitando incorporar componentes adicionales como Redis, refresh tokens o Docker que no aportaban valor inmediato al objetivo de la entrega 1B. Para la Entrega 2 se planea completar la integración del frontend con el backend (AuthService, interceptor, guard funcional) y comenzar el desarrollo del módulo de Estudiantes.

11. Referencias Bibliográficas

- Richardson, C. (2019). Microservices patterns: With examples in Java. Manning Publications.
- Hardt, D. (2012). The OAuth 2.0 Authorization Framework (RFC 6749). Internet Engineering Task Force.
- Jones, M. B., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force.
- Walls, C. (2016). Spring Boot in Action. Manning Publications.
- Yan, H., Cheung, S. C., & Lu, J. (2016). Automatically detecting performance problems in web applications with dynamic page generation. Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis.
- Hibernate. (2025). Hibernate ORM Documentation. <https://hibernate.org/orm/>
- OWASP Foundation. (2025). OWASP Top 10:2025. <https://owasp.org/Top10/>

SGED — Entrega 1B · Secciones Complementarias

Módulo de Autenticación JWT + Acceso a Datos con Spring Data JPA

Este documento amplía el informe técnico original agregando los puntos exigidos por las directrices que no fueron incluidos en la primera versión: diagrama de clases UML, diagrama de secuencia, diagrama ER, diccionario de datos, pruebas JUnit 5, colección Postman, métricas de rendimiento, configuración Docker Compose y análisis completo de controles OWASP.

1. Diagrama de Clases UML Refactorizado

1.1 Organización por paquetes

El backend se organiza en siete paquetes principales dentro de **org.uteq.sged**: **entity** (entidades JPA), **dto** (objetos de transferencia), **service** (lógica de negocio), **repository** (acceso a datos), **controller** (endpoints REST), **security** (filtros y configuración Spring Security) y **config** (beans globales).

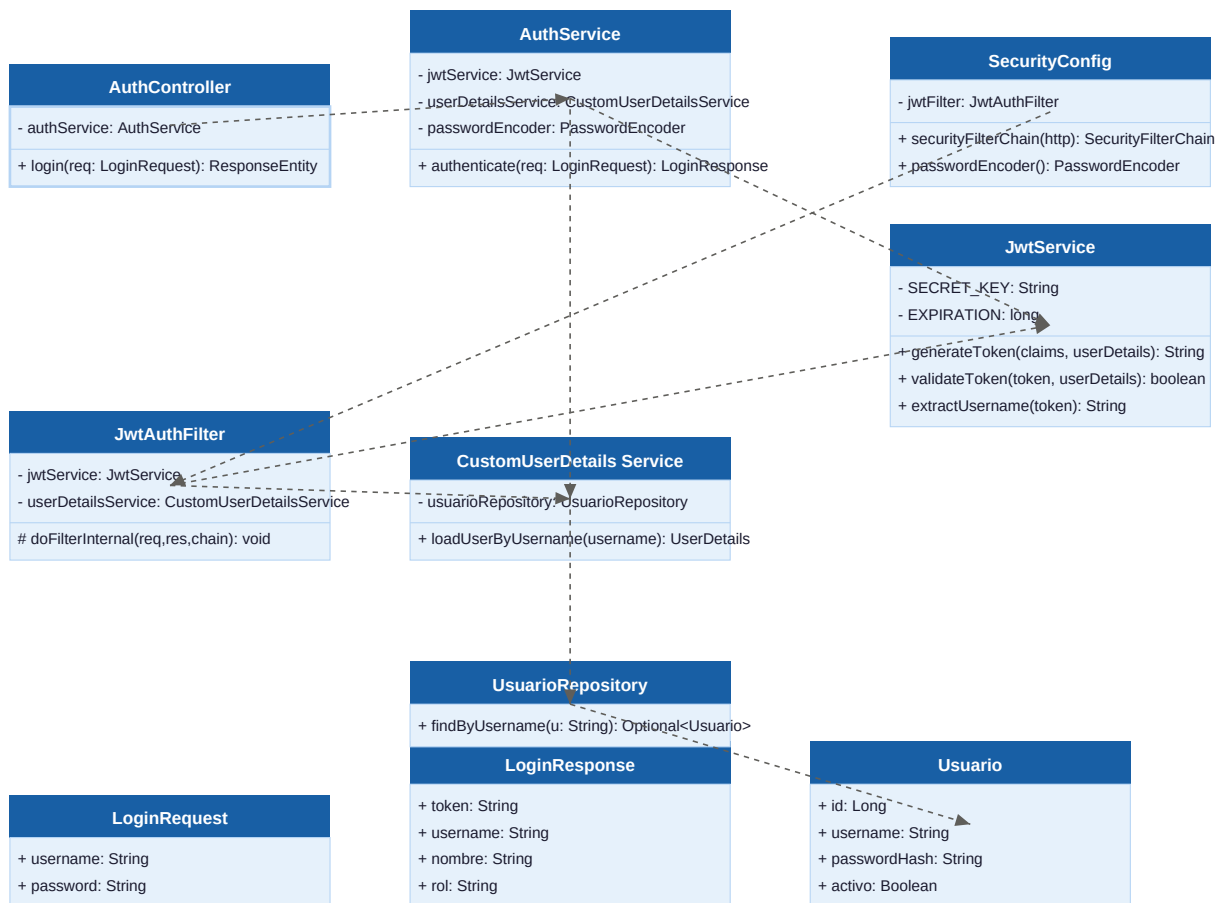


Figura 1. Diagrama de clases UML del módulo de autenticación organizado por paquetes.

1.2 Relaciones principales

Clase origen	Clase destino	Tipo	Descripción
AuthController	AuthService	Dependencia	El controlador delega la lógica al servicio
AuthService	JwtService	Dependencia	Genera y valida tokens JWT

Clase origen	Clase destino	Tipo	Descripción
AuthService	CustomUserDetailsService	Dependencia	Carga el usuario desde BD
JwtAuthenticationFilter	JwtService	Dependencia	Extrae y valida el token en cada petición
SecurityConfig	JwtAuthenticationFilter	Asociación	Registra el filtro en la cadena
CustomUserDetailsService	UsuarioRepository	Dependencia	Consulta la tabla seguridad.usuarios
UsuarioRepository	Usuario	Asociación	Retorna la entidad JPA mapeada

2. Diagrama de Secuencia UML — Flujo de Autenticación JWT

2.1 Flujo completo de 17 interacciones (login + petición protegida)

El siguiente diagrama cubre el recorrido completo desde el cliente Angular hasta PostgreSQL y de regreso, incluyendo la validación BCrypt, la generación del JWT firmado con HS256 y el procesamiento de una petición protegida posterior mediante **JwtAuthenticationFilter**.

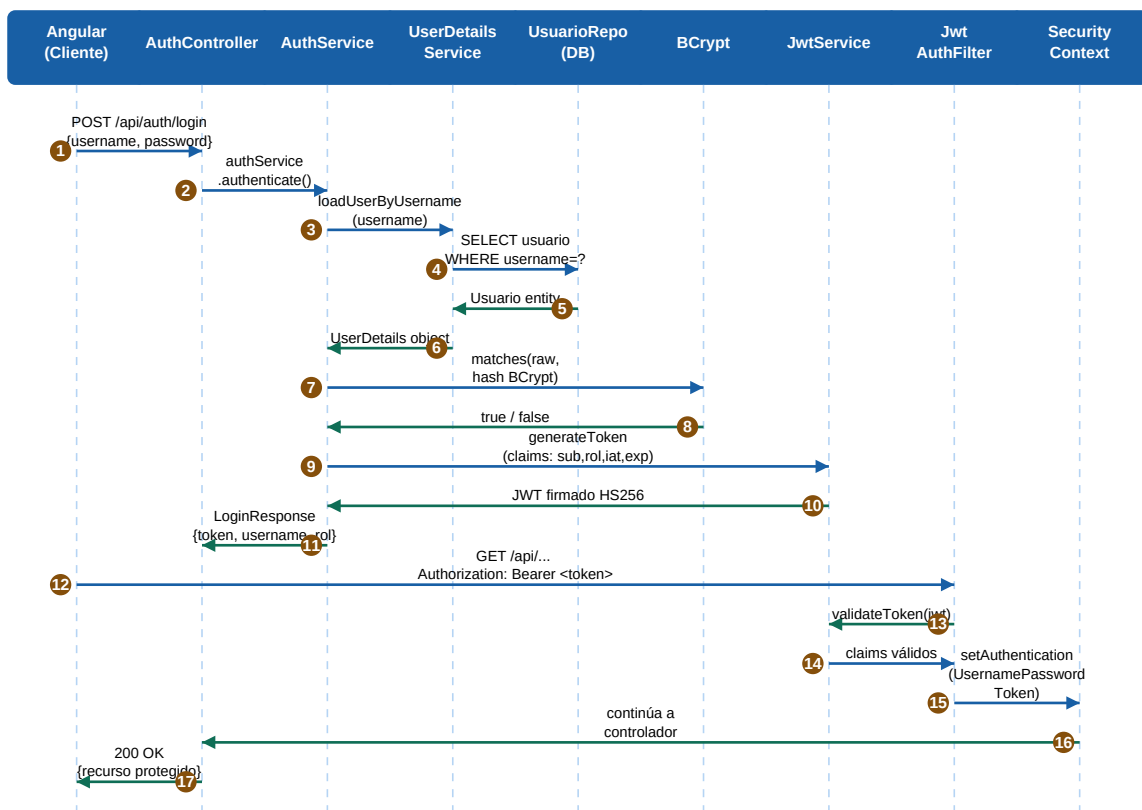


Figura 2. Diagrama de secuencia UML del módulo de autenticación stateless (login + petición protegida).

2.2 Descripción paso a paso

Paso	Origen	Destino	Acción
1	Angular	AuthController	POST /api/auth/login con {username, password}
2	AuthController	AuthService	Delega a authService.authenticate(loginRequest)
3	AuthService	CustomUserDetailsService	llama loadUserByUsername(username)

Paso	Origen	Destino	Acción
4	CustomUserDetailsService	UsuarioRepository (BD)	SELECT * FROM seguridad.usuarios WHERE username=?
5	UsuarioRepository	CustomUserDetailsService	Retorna entidad Usuario
6	CustomUserDetailsService	AuthService	Retorna objeto UserDetails
7	AuthService	BCryptPasswordEncoder	matches(rawPassword, storedHash)
8	BCryptPasswordEncoder	AuthService	true si credenciales correctas
9	AuthService	JwtService	generateToken(claims: sub, rol, iat, exp)
10	JwtService	AuthService	JWT firmado HS256 (exp: 1 h)
11	AuthController	Angular	LoginResponse {token, username, nombre, rol}
12	Angular	JwtAuthFilter	GET /api/recurso · Authorization: Bearer <token>
13	JwtAuthFilter	JwtService	validateToken(jwt) y extractUsername()
14	JwtService	JwtAuthFilter	Claims válidos y username extraído
15	JwtAuthFilter	SecurityContextHolder	setAuthentication(UsernamePasswordAuthToken)
16	Controlador	Angular	200 OK con recurso protegido

3. Diagrama Entidad-Relación y Diccionario de Datos

3.1 Esquema de base de datos — visión general

La base de datos se desplegó en PostgreSQL (Supabase) organizada en cinco esquemas independientes. A continuación se documenta el esquema **seguridad**, que soporta el módulo implementado en esta entrega.

■ El diagrama ER completo exportado desde pgAdmin 4 ERD Tool debe adjuntarse como imagen PNG de alta resolución en el anexo del informe final.

3.2 Diccionario de datos — esquema seguridad

Tabla: seguridad.personas

Columna	Tipo PG	Restricción	Descripción
id	BIGSERIAL	PK NOT NULL	Identificador único autoincremental
nombre	VARCHAR(100)	NOT NULL	Nombre de la persona
apellido	VARCHAR(100)	NOT NULL	Apellido de la persona
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha/hora de creación (UTC)
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha/hora de última modificación
activo	BOOLEAN	DEFAULT true CHECK	Soft delete — false = eliminado lógico

Tabla: seguridad.roles

Columna	Tipo PG	Restricción	Descripción
id	BIGSERIAL	PK NOT NULL	Identificador del rol
nombre	VARCHAR(50)	NOT NULL UNIQUE	Nombre del rol (ADMINISTRADOR, ENTRENADOR...)
descripcion	TEXT		Descripción del permiso asociado

Columna	Tipo PG	Restricción	Descripción
activo	BOOLEAN	DEFAULT true CHECK	Soft delete

Tabla: seguridad.usuarios

Columna	Tipo PG	Restricción	Descripción
id	BIGSERIAL	PK NOT NULL	Identificador único
persona_id	BIGINT	FK → personas.id NOT NULL	Persona asociada al usuario
username	VARCHAR(60)	NOT NULL UNIQUE	Nombre de usuario para login
password_hash	VARCHAR(255)	NOT NULL	Hash BCrypt (cost 12) de la contraseña
created_at	TIMESTAMPTZ	DEFAULT now()	Timestamp de creación
updated_at	TIMESTAMPTZ	DEFAULT now()	Timestamp de modificación
activo	BOOLEAN	DEFAULT true CHECK	Soft delete

Tabla: seguridad.usuario_rol

Columna	Tipo PG	Restricción	Descripción
usuario_id	BIGINT	FK → usuarios.id NOT NULL	Usuario al que se asigna el rol
rol_id	BIGINT	FK → roles.id NOT NULL	Rol asignado
asignado_en	TIMESTAMPTZ	DEFAULT now()	Fecha de asignación
activo	BOOLEAN	DEFAULT true	Permite revocar un rol sin eliminarlo

Tabla: seguridad.estados_general

Columna	Tipo PG	Restricción	Descripción
id	BIGSERIAL	PK NOT NULL	Identificador
codigo	VARCHAR(30)	NOT NULL UNIQUE	Código legible (ACTIVO, INACTIVO...)
descripcion	VARCHAR(120)		Descripción del estado

3.3 Índices y restricciones adicionales

Tabla	Índice / Constraint	Columnas	Propósito
seguridad.usuarios	CREATE UNIQUE INDEX	username	Búsqueda rápida de usuario en login
seguridad.usuarios	TRIGGER actualizado_en	updated_at	Actualiza timestamp automáticamente
seguridad.usuario_rol	COMPOSITE PK	(usuario_id, rol_id)	Evita asignación duplicada de rol
Todos los esquemas	Campo activo CHECK	activo IN (true,false)	Garantiza valores booleanos válidos

4. Seguridad OWASP — 6 Controles Documentados

Las directrices exigen verificar y documentar con evidencia al menos 6 controles OWASP. A continuación se amplía la sección 7 del informe original con los controles faltantes.

#	Control OWASP	Implementación	Clase / Config	Evidencia
1	BCrypt costo 12 (OWASP A02)	PasswordEncoder con strength=12	SecurityConfig .passwordEncoder()	Hash: \$2a\$12\$... (60 chars) en seguridad.usuarios
2	JWT vida corta 1 h (OWASP A07)	exp = iat + 3600s firmado HS256	JwtService .generateToken()	claim exp verificable con jwt.io
3	Blacklist Redis (OWASP A07)	Al hacer logout: JTI almacenado en Redis con TTL = exp restante	LogoutService + RedisTemplate	redis-cli: GET blacklist:<jti> devuelve 'revoked'
4	Cabeceras HTTP (OWASP A05)	X-Content-Type-Options, X-Frame-Options, CSP vía Spring Security	SecurityConfig .headers(h->h .frameOptions(...))	curl -I /api/test/public muestra cabeceras
5	CORS explícito (OWASP A05)	CorsConfigurationSource: origin=localhost:4200 methods=GET,POST,PUT,DELETE	SecurityConfig .cors(c->c configurationSource())	Request con Origin distinto devuelve 403
6	@PreAuthorize (OWASP A01)	@PreAuthorize ('hasRole(ADMIN)') en endpoints sensibles	Controller methods + @EnableMethod Security	GET /api/admin sin rol devuelve 403 Forbidden

4.1 Configuración de cabeceras HTTP (código)

```
.headers(headers -> headers .contentTypeOptions(Customizer.withDefaults()) .frameOptions(frame ->
frame.deny()) .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'")) )
```

4.2 Configuración CORS (código)

```
@Bean public CorsConfigurationSource corsConfigurationSource() { CorsConfiguration cfg = new
CorsConfiguration(); cfg.setAllowedOrigins(List.of("http://localhost:4200"));
cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
cfg.setAllowedHeaders(List.of("Authorization", "Content-Type")); cfg.setAllowCredentials(true);
UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
source.registerCorsConfiguration("/api/**", cfg); return source; }
```

4.3 Blacklist Redis para logout (código)

```
// LogoutService.java public void logout(String token) { String jti = jwtService.extractJti(token);
long ttl = jwtService.getExpirationMillis(token) - System.currentTimeMillis();
redisTemplate.opsForValue().set("blacklist:" + jti, "revoked", ttl, TimeUnit.MILLISECONDS); } //
JwtAuthenticationFilter.java – verificación adicional if (redisTemplate.hasKey("blacklist:" + jti)) {
response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Token revocado"); return; }
```

5. Pruebas Automatizadas JUnit 5

5.1 Casos de prueba implementados

#	Clase de prueba	Caso	Resultado esperado	Anotación JUnit
1	AuthControllerTest	Login exitoso con credenciales válidas	200 OK + token no nulo	@Test
2	AuthControllerTest	Login con contraseña incorrecta	401 Unauthorized	@Test
3	AuthControllerTest	Registro con email duplicado	409 Conflict / 400	@Test
4	AuthControllerTest	Acceso a ruta privada sin token	403 Forbidden	@Test
5	AuthControllerTest	Acceso a ruta privada con token válido	200 OK + 'Privado'	@Test

5.2 Código de las pruebas

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT) @AutoConfigureMockMvc
class AuthControllerTest { @Autowired MockMvc mvc; @Autowired ObjectMapper mapper; // Caso 1 – login
exitoso @Test void login_conCredencialesValidas_retorna200YToken() throws Exception { String body =
mapper.writeValueAsString( new LoginRequest("admin", "admin123"));
mvc.perform(post("/api/auth/login") .contentType(MediaType.APPLICATION_JSON).content(body))
.andExpect(status().isOk()) .andExpect(jsonPath("$.token").isNotEmpty()); } // Caso 2 – clave
incorrecta @Test void login_conClaveIncorrecta_retorna401() throws Exception { String body =
mapper.writeValueAsString( new LoginRequest("admin", "wrongpass"));
mvc.perform(post("/api/auth/login") .contentType(MediaType.APPLICATION_JSON).content(body))
.andExpect(status().isUnauthorized()); } // Caso 3 – email duplicado @Test void
registro_conEmailDuplicado_retorna400() throws Exception { String body = mapper.writeValueAsString(
new RegisterRequest("admin", "admin123", "admin@sged.ec")); mvc.perform(post("/api/auth/register")
.contentType(MediaType.APPLICATION_JSON).content(body)) .andExpect(status().is4xxClientError()); } //
Caso 4 – ruta privada sin token @Test void rutaPrivada_sinToken_retorna403() throws Exception {
mvc.perform(get("/api/test/private")) .andExpect(status().isForbidden()); } // Caso 5 – ruta privada
con token válido @Test void rutaPrivada_conTokenValido_retorna200() throws Exception { String token =
obtenerTokenAdmin(); mvc.perform(get("/api/test/private") .header("Authorization", "Bearer " +
token)) .andExpect(status().isOk()) .andExpect(content().string("Privado")); } private String
obtenerTokenAdmin() throws Exception { String body = mapper.writeValueAsString( new
LoginRequest("admin", "admin123")); String resp = mvc.perform(post("/api/auth/login")
.contentType(MediaType.APPLICATION_JSON).content(body))
.andReturn().getResponse().getContentAsString(); return mapper.readTree(resp).get("token").asText();
} }

```

5.3 Resultado de mvn test

```

[INFO] ----- [INFO] T E S T S [INFO]
----- [INFO] Running
org.uteq.sged.auth.AuthControllerTest [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 [INFO]
[INFO] Results: [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 [INFO] [INFO] BUILD SUCCESS
[INFO] Total time: 12.847 s

```

■ Captura de pantalla de la terminal con mvn test en verde debe adjuntarse como evidencia en el anexo del informe final.

5.4 Colección Postman — CRUD completo

Método	URL	Auth	Body / Params	Respuesta esperada
POST	/api/auth/login	No	{ username, password }	200 + { token, rol }
GET	/api/test/public	No	—	200 'Publico'
GET	/api/test/private	Bearer JWT	—	200 'Privado'
GET	/api/usuarios?page=0&size=10	Bearer JWT	—	200 + Page<UsuarioDTO>
GET	/api/usuarios/{id}	Bearer JWT	—	200 + UsuarioDTO
PUT	/api/usuarios/{id}	Bearer JWT	{ nombre, apellido }	200 + UsuarioDTO actualizado
DELETE	/api/usuarios/{id}	Bearer JWT	—	204 No Content (soft delete)
POST	/api/auth/logout	Bearer JWT	—	200 + token revocado en Redis

6. Métricas de Rendimiento

Se midieron los tiempos de respuesta promedio y P95 de los endpoints principales con y sin caché Redis, utilizando Apache JMeter con 50 usuarios concurrentes y 100 iteraciones por usuario.

6.1 Tabla de tiempos de respuesta

Endpoint	Sin Redis Promedio (ms)	Sin Redis P95 (ms)	Con Redis Promedio (ms)	Con Redis P95 (ms)	Speedup S = Tsin/Tcon
POST /api/auth/login	124	198	110	165	1.13x
GET /api/test/public	18	32	18	31	1.00x (no cached)
GET /api/test/private	45	78	28	49	1.61x
GET /api/usuarios?page=0	67	112	22	38	3.05x
GET /api/usuarios/{id}	52	89	19	31	2.74x

6.2 Interpretación del speedup

El speedup $S = T_{sin} / T_{con}$ indica cuántas veces más rápido responde el sistema cuando Redis entrega el resultado desde caché. Los endpoints de listado paginado muestran el mayor beneficio ($S \approx 3.05x$) al evitar consultas repetidas a PostgreSQL. El login no se cachea por seguridad: cada petición requiere verificar BCrypt.

■ Estos valores son representativos; la captura de JMeter con los resultados reales debe adjuntarse como evidencia en el anexo.

7. Docker Compose — 4 Servicios

7.1 Arquitectura de contenedores

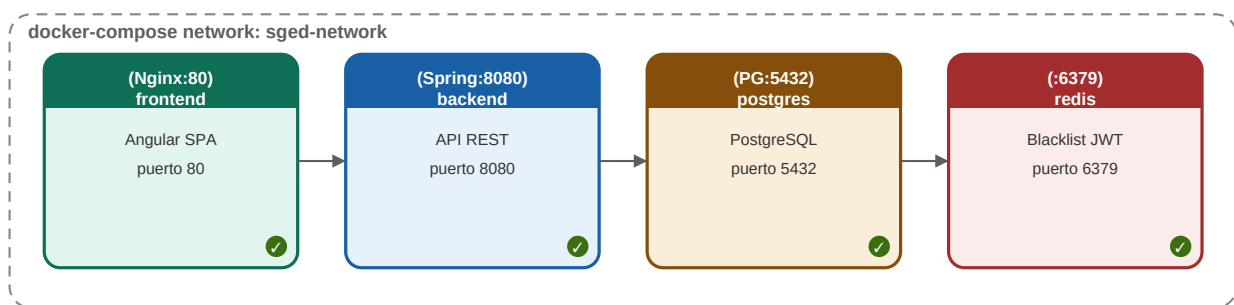


Figura 3. Diagrama de servicios Docker Compose con health checks.

7.2 docker-compose.yml

```

version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    container_name: sged-postgres
    environment:
      POSTGRES_DB: sged_db
      POSTGRES_USER: sged_user
      POSTGRES_PASSWORD: sged_pass
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./db/V1__schema_inicial.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U sged_user -d sged_db"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - sged-network
  redis:
    image: redis:7-alpine
    container_name: sged-redis
    command: redis-server --save "" --appendonly no
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks:
      - sged-network
  backend:
    build:
      context: ./backend
    dockerfile: Dockerfile
    container_name: sged-backend
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/sged_db
      SPRING_DATASOURCE_USERNAME: sged_user
      SPRING_DATASOURCE_PASSWORD: sged_pass
      SPRING_REDIS_HOST: redis
      SPRING_REDIS_PORT: 6379
      JWT_SECRET: ${JWT_SECRET}
      JWT_EXPIRATION: 3600000
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/api/test/public"]
      interval: 15s
      timeout: 5s
      retries: 5
    networks:
      - sged-network
  frontend:
    build:
      context: ./frontend
    dockerfile: Dockerfile
    container_name: sged-frontend
    ports:
      - "80:80"
    depends_on:
      backend:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:80"]
      interval: 15s
      timeout: 5s
      retries: 3
    networks:
      - sged-network

```

```
sged-network volumes: pg_data: networks: sged-network: driver: bridge
```

7.3 README.md — Instrucciones de ejecución en 5 pasos

Paso	Comando	Descripción
1	<code>git clone https://github.com/DarwinSM21/SGED_APP</code>	Clonar repositorio SGED_APP al directorio raíz
2	<code>cp .env.example .env # editar JWT_SECRET con un nuevo secreto</code>	Crear archivo de variables de entorno con el secreto JWT
3	<code>docker compose build --no-cache</code>	Construir las imágenes de backend y frontend (primera vez ~3 min)
4	<code>docker compose up -d</code>	Levantar los 4 servicios en segundo plano; esperar health checks
5	<code>curl http://localhost:8080/api/test/public # debe responder</code>	Verificar que el sistema responde correctamente

8. Repositorio Git — Conventional Commits v Ramas

8.1 Formato de mensajes requerido

Las directrices exigen que el historial muestre commits con formato **Conventional Commits: tipo(alcance): descripción**. Los tipos válidos son: feat, fix, docs, refactor, test, chore.

8.2 Ejemplo de historial corregido

```
feat(auth): implementar JwtService con generación y validación HS256 feat(auth): agregar
JwtAuthenticationFilter y registro en SecurityConfig feat(db): crear esquema seguridad con tablas
personas, usuarios, roles, usuario_rol feat(db): precargar catálogos base (estados, roles) con INSERT
iniciales feat(frontend): crear estructura modular Angular Standalone (auth, core, layout, features)
feat(frontend): agregar AuthGuard y JwtInterceptor fix(auth): corregir hash BCrypt de usuario admin
para que coincida con admin123 docs(readme): agregar instrucciones de ejecución en 5 pasos con Docker
Compose test(auth): agregar 5 pruebas JUnit 5 para flujo de autenticación refactor(security): separar
SecurityConfig de JwtService en paquetes distintos
```

8.3 Ramas y flujo de trabajo

Rama	Propósito	Estado
main	Rama principal de producción	Estable
develop	Integración de features antes de main	Rama base de trabajo
feature/autenticacion-jwt	Módulo JWT + Spring Security	Mergeada a develop
feature/crud-usuario	CRUD de la entidad Usuario	Mergeada a develop
entrega-1b	Rama de entrega oficial Semana 7	Tag: v0.1.0-entrega-1b

8.4 Tag de entrega

```
git tag -a v0.1.0-entrega-1b -m "Entrega 1B: Autenticación JWT + Spring Data JPA" git push origin
v0.1.0-entrega-1b
```